Privacy-Preserving Stream Joins

Kajetan Maliszewski
RelationalAI
kajetan.maliszewski@relational.ai

Ioannis Demertzis
UC Santa Cruz
idemertz@ucsc.edu

Volker Markl
BIFOLD & TU Berlin
volker.markl@tu-berlin.de

***Abstract*—A growing number of cloud applications process sensitive streaming data. Yet, the existing stream processing systems lack privacy-preserving operators, particularly joins. Despite using trusted hardware, these systems are vulnerable to access pattern attacks when executing on cloud infrastructure.**

We formally define the unexplored problem of oblivious stream joins in secure environments and introduce the first configurable leakage framework for such operators over encrypted data. Our framework defines five leakage functions that enable new privacy-performance tradeoffs in unbounded environments. We address the lack of support for secure stream join queries and propose two families of oblivious algorithms for non-foreign key and foreign-key joins: index-based, using ORAM indices, and computation-based, using oblivious primitives. Our solutions achieve this with new oblivious tools: OBLIWIND, a B-Tree-based doubly-oblivious window management data structure, and OAPPEND, an append primitive that maintains item order without revealing access patterns. We demonstrate empirically that our fastest foreign-key stream join achieves performance within an order of magnitude of insecure baselines while offering strong privacy guarantees. Compared to ORAM-based approaches, we achieve a 3- to 6-orders-of-magnitude performance improvement, making privacy-preserving stream joins practical for real-world deployments.

I. INTRODUCTION

Modern cloud applications frequently handle real-time streaming data, such as healthcare devices analyzing physiological signals [1] and IoT devices tracking user locations. While many organizations have implemented privacy measures for static data [2]–[5], these measures do not address the challenges of stream processing, such as continuous queries. Moreover, existing stream processing systems (SPSs) [6]–[8] lack privacy-preserving mechanisms, particularly when joining multiple streams [9], which remains among the most complex and resource-intensive tasks.

Secure Streaming: In cloud computing, data must be protected in all states: *at-rest*, *in-transit*, and *in-use* [10]. However, the existing SPSs [6], [7] protect only the first two states (via encryption and transport layer security), exposing data *in-use* to malicious observers (e.g., cloud administrators). Moreover, despite the three-state protection, query pattern leakage [11]–[13] and side-channel attacks [14]–[16] reveal additional information about the data. For example, consider GPS devices that provide real-time map services by joining stream location data with map data. Even with end-to-end encryption, malicious observers can monitor join patterns via memory accesses [17] - which encrypted location records access which regions and at what frequency. This metadata leakage reveals sensitive information: high join frequencies expose popular locations, join patterns identify users with comparable behaviors, and cross-stream patterns reveal user groups. Thus, while raw data remain encrypted, query execution metadata expose the very patterns encryption aims to protect. Therefore, there is a clear need for privacy-preserving SPSs that protect users' activities and behaviors.

Oblivious Computation: Many privacy-oriented solutions use Trusted Execution Environments (TEEs), hardware-isolated computing environments, as fundamental building blocks for data confidentiality [18]–[20]. TEEs work by creating encrypted memory regions accessible only by the CPU. Yet, as shown in the previous example and related work [11]–[13], [21]–[23], TEEs alone leak sensitive information from their encrypted memory. To solve this problem, practitioners use *oblivious* computation, which meticulously tracks access patterns [24]–[26], ensuring data independence. For example, *Signal* (an instant messaging service) operates an oblivious contact discovery [27], and *TikTok* utilizes an oblivious friends-matching mechanism [5]. Nonetheless, no existing system supports oblivious stream joins.

Limitations of Existing Works: While oblivious static joins have received significant attention [18], [19], [28], [29], no prior work has addressed oblivious stream joins in the *trusted hardware* model. Yet, naively using static joins in streaming leaks intermediate cardinalities. Therefore, we benchmarked two existing stream join algorithms - insecure Symmetric Hash Join (*SHJ*) and TEE-based Nested-Loop stream Join with worst-case padding (*NLJ-L4*). While *SHJ* is a popular index-based stream join algorithm [7], [30], [31], *NLJ-L4* is a strawman solution for stream joins that hides access patterns [32]. The results (§ VII-A) show a three orders of magnitude *performance* gap between the algorithms caused by *NLJ-L4*'s high complexity. Additionally, *SHJ* and *NLJ-L4* represent opposite sides of the privacy scale, uncovering a *privacy* gap in existing work. Moreover, in multi-join query plans, the overhead from *NLJ-L4*'s padding exhibits multiplicative growth, saturating downstream operators with dummy tuples. We strongly believe that users would benefit from stream joins with a tunable privacy-performance tradeoff. Tunable privacy has been explored in various data processing scenarios [33]–[35], but it has not yet been applied to stream joins. Therefore, we reveal a clear need for stream join algorithms that improve both *SHJ*'s security and *NLJ-L4*'s performance.

Challenges of Oblivious Stream Joins: Designing oblivious stream joins presents several challenges. First, achieving full obliviousness requires a costly cross join (i.e., Cartesian product) to hide output cardinality, yet no intermediate stream-

ing leakage models exist. This reveals a gap in security models, which do not align with the streaming paradigm. Second, stream applications prioritize low latency, often measured as throughput, and eager execution. However, existing oblivious techniques, largely based on Oblivious RAM (ORAM) [27], [36], [37] or assuming oblivious memory [18], [19], suffer from high latency [38] and complex parallelization [39]. TEEs, though widely adopted, lack native oblivious memory [40], rendering many techniques impractical. Third, stream joins operate over windows. It is unclear how to manage these efficiently while maintaining strong privacy. Overall, the privacy-performance tradeoff, already a challenge in static settings, is amplified in streaming, where real-time constraints and continuous updates make current approaches [41]–[43] unsuitable.

Privacy-Preserving Stream Joins: We take the first step towards fully private stream processing on untrusted infrastructure by introducing *oblivious stream joins* – a critical, yet unexplored, building block for secure streaming systems. Existing SPSs lack support for joins that protect against access-pattern leakage, leaving users vulnerable even when leveraging trusted hardware. To address this challenge, we propose a formal streaming security framework, consisting of five leakage functions ($\mathcal{L}_0$-$\mathcal{L}_4$), which represent a spectrum of privacy guaranties: from weak confidentiality to strong encryption with obliviousness. Our framework enables fine-grained control over the privacy-performance tradeoff, filling a critical gap in stream processing security. We instantiate the framework with the first suite of oblivious stream join algorithms, spanning index-based (SHJ) and computation-based (OCA) families. Our index-based joins rely on OBLIWIND, a novel B-Tree-backed, doubly oblivious sliding window data structure that supports efficient deletions and timestamp ordering. Moving beyond ORAM's limitations, we introduce OAPPEND, a lightweight oblivious primitive for ordered appends used by the OCA joins. Our flagship join, *FK-MERG-L4*, achieves full obliviousness while maintaining throughput within an order of magnitude of insecure baselines. Through extensive evaluation, we demonstrate that our algorithms bridge the performance-privacy gap between insecure *SHJ* and the strawman *NLJ-L4*, outperforming ORAM-based solutions by 3-6 orders of magnitude. Our work lays the foundation for practical and secure streaming systems that protect not only the data content, but also the behavioral patterns emerging from that data. In summary, we make the following contributions:

(1) We formulate the previously unexplored problem of oblivious joins in unbounded environments (§ III).
(2) We propose a formal security framework for stream joins based on a novel leakage taxonomy (§ IV).
(3) We contribute two foundational stream processing primitives: OBLIWIND, an oblivious window structure, and OAPPEND, an oblivious append that maintains sorted collections.
(4) We present the first two families of oblivious stream joins: index-based (§ V), and oblivious-primitive-based (§ VI).
(5) We evaluate all algorithms, demonstrating significant improvement over baselines and the strawman solution (§ VII).

II. PRELIMINARIES

We now provide the necessary background. We discuss the stream processing and security foundations (§ II-A), the threat model (§ II-B), and related work (§ II-C).

A. Background

Stream processing analyzes unbounded, continuous data in real-time, providing millisecond-level insights for applications such as financial transactions, social media feeds, and IoT sensors. We adopt the terminology from Verwiebe et al. [44]. A data stream $\mathcal{S}$ is an infinite sequence of tuples containing a timestamp τ, a key, and a payload. We define finite subsets of streams as windows $\mathcal{W}$ of size w, enabling operations that would otherwise be impossible to define, such as joins and aggregations. We focus on *sliding windows*, i.e., windows that contain tuples from an earlier time until the most recent tuple. Sliding windows support two modes: *time-based*, with tuples from the last τ time units, and *count-based*, containing the last k tuples. Stream tuples are processed *tuple-at-a-time* or in *(micro-)batches*. In the former, a tuple is joined as soon as it arrives, resulting in low latency and higher processing costs [7]. In the latter, tuples form batches that are joined at fixed intervals [6], [45]–[47], achieving higher average throughput but also higher latency. Streams can be consumed via push- and pull-based models. Typically, the pull-based model is used for incremental view maintenance [48] or dashboards, while the push-based model, which is our focus, for real-time cases [49]. Modern SPSs [7], [45] protect data via encryption. Despite this mechanism, user data remain vulnerable to side-channel attacks, including memory access patterns [21], [50], control flow execution paths [14], or output rates [51]. To mitigate these risks, SPSs should implement countermeasures, such as oblivious processing, constant-time algorithms, and padding. Our paper addresses this gap.

Stream join operators continuously combine rows from multiple streams on a join predicate, producing incremental results. Unlike relational joins [52] that combine static tables, stream joins process unbounded data using windows to delineate the working dataset. A *foreign key* join (FK) occurs when one stream contains unique keys (e.g., unique customers referenced by many orders). In contrast, a *non-foreign key* join (non-FK) is a generic join where streams may contain duplicates (e.g., joining on matching dates). Non-FK are more complex due to lacking relationship restrictions. The join type determines worst-case padding (i.e., the maximum output size): FK joins pad to the foreign key window size, while non-FK joins pad to the windows' Cartesian product, significantly affecting the performance. Finally, binary joins can be represented as bipartite *join graphs* [17], where tuples form two disjoint vertex sets and edges represent join matches.

Symmetric hash join (SHJ) is the primary join algorithm used in stream processing. SHJ maintains hash tables for both streams and performs the three-step procedure [53]. For each incoming tuple, it: (i) adds the tuple to its stream's hash table, (ii) probes the opposite stream's table, and (iii) invalidates

expired tuples. SHJ is often the only join implementation in SPSs [7], [8], [45] thanks to its $O(1)$ amortized complexity.

Nested-loop join (NLJ) is a relational join that uses nested iterations through the joined tables. Despite quadratic performance, all major databases implement it due to simplicity and well-understood cost. NLJ executes privately (i.e., all data/-code access patterns remain hidden) thanks to a deterministic sequential scan per tuple. In the MPC setup, NLJ is often used as a secure join [54]–[56]. NLJ also achieves full privacy when executed inside a TEE with worst-case padding. As a baseline stream join derived from prior MPC work, *NLJ-L4* (Section VII-A) sequentially scans the opposite window for each incoming tuple. It emits join results if it finds a match and, for full privacy, dummy tuples in case of no match.

Trust models. Secure computation includes Multi-Party Computation (MPC) [57], where multiple data owners compute without sharing inputs, and outsourced computation, where computation is offloaded to untrusted parties. For single-cloud solutions, we focus on outsourced computation with two trust models: *trusted hardware* (secure co-processors/CPUs in untrusted machines) [18], [19], [58], [59] or *trusted proxy* (separating computation from storage) [28], [39], [60]–[64]. While both protect data during computation, *trusted hardware* is more relevant to public cloud as it provides a uniform environment and is supported by all major cloud providers. We assume the *trusted hardware* model.

Trusted Execution Environments (TEEs) , also called *enclaves* [65]–[67], are CPU-isolated execution environments that provide code and data confidentiality and integrity. Intel SGX [67] is the most widely deployed TEE in the public cloud. SGX encrypts data in isolated Enclave Page Cache (EPC) memory, with SGXv2 supporting up to hundreds of GBs. SGX persistently stores encrypted data outside enclaves via *sealing*, while also verifying code integrity and validating hardware configuration via *remote attestation*.

Oblivious processing [17], [68], [69] ensures that data/-code access patterns and intermediate results remain hidden. An algorithm is oblivious if its access patterns are computationally indistinguishable across all inputs of the same size, i.e., a polynomial-time adversary cannot distinguish the memory address sequence accessed during computation for any two equal-length inputs. Obliviousness is crucial when offloading computation to untrusted parties, such as cloud providers, where it mitigates most of the side-channel attacks (e.g., access pattern, power consumption, or timing attacks) [15]. Such computation has been applied to distributed analytics [18], relational databases [19], and graph processing [20], but not yet to stream processing. *Double-obliviousness* [36] extends these guarantees to both TEE-protected and external memory.

Oblivious sort sorts an array without revealing information about the process or the data. Two approaches are sorting networks (e.g., bitonic sort [70]) that use deterministic oblivious swaps, and bucket oblivious sort [71], combining oblivious shuffle with non-oblivious comparison sort. Bitonic sort is popular due to its practical efficiency and simple implementation. Despite its $O(n \log^2 n)$ running time being higher than other constructions (e.g., AKS [72] achieves $O(n \log n)$), it remains competitive due to small constants.

Oblivious compaction [73], [74] permutes elements, placing marked items before unmarked ones. Oblivious compaction is often used to filter out "dummy" elements introduced to hide previous memory operations. While naively sorting on marked flags yields correct results, Sasy et al. [75] proposed a faster, $O(NlogN)$ algorithm as a deterministic set of oblivious conditional swaps. Their method splits the input array into left and right sides, recursively compacting the left side, while computing the number of "marked" items in it. It then compacts the right side and conditionally swaps pairwise with the remaining elements from the left side.

Oblivious shuffle produces a random permutation without revealing access patterns. While sorting on random identifiers works, specialized algorithms achieve higher performance. ORSHUFFLE [75], despite $O(nlog^2n)$ complexity, is currently the fastest oblivious shuffle, using random split via oblivious compaction before recursively shuffling each half.

Oblivious RAM (ORAM) [25], [26] enables clients to access data stored on untrusted servers without revealing access patterns. Unlike traditional encryption, which only hides data values, ORAM shuffles and re-encrypts data after each operation, making repeated accesses appear random and unpredictable. Path ORAM [76] and Circuit ORAM [77] are among the widely used constructions. ORAMs are still facing numerous practical challenges, particularly in streaming, including high latency and limited parallelization [78].

Oblivious maps [20], [27], [36], [37] typically use oblivious AVL trees [79] inside TEEs with ORAM-stored nodes [24]. While AVL's self-balancing property is beneficial, achieving full obliviousness requires worst-case padding for all operations. This makes deletions particularly expensive due to greater constants than insertions. While [37] reports deletions as being $5\times$ more expensive than insertions, many works [20], [36] disregard them completely.

B. Threat Model

We combine TEEs and obliviousness in a workflow similar to prior works [18]–[20], [28], [36], [37], [41]. We protect against adversaries with OS and infrastructure control who perform memory snapshots, monitor access patterns, network communication, and CPU instructions. However, they cannot break into the TEE. As in [18], [19], [28], [41], we consider certain input stream information public, specifically: window sizes and tuple counts (which reflect input/output sizes in static joins), as well as timestamps. Thus, the attacker can reconstruct the windows using public information. While mitigation techniques exist (e.g., fixed-size batching [63] and traffic obfuscation [80]), they incur very high costs. Hiding this metadata is left to the stream provider and our framework supports both padded and unpadded streams. Further, we prevent the attacker from learning the values of individual tuples, and we formally define which elements of the join graph leak using leakage functions. The adversary tries to learn these facts

by observing memory and code access patterns. We comply with this threat model by executing operators within TEEs to ensure data confidentiality, integrity, and isolation. We design data-oblivious algorithms that hide memory access patterns and ensure deterministic code execution through code padding and dummy operations. We implemented our solution on Intel SGXv2 [67], the most popular cloud hardware enclave. Other TEEs [65] would serve equally well.

C. Related Work

Privacy-preserving data processing has evolved from weak encryption schemes (i.e., DET- and order-preserving encryption), used by CryptDB [81] and Always Encrypted [82]. Yet, a line of attacks [12], [13] has shown that they fail to provide the promised privacy. This led to TEE-based solutions: EnclaveDB [58] is a TEE database engine, while Cipherbase [83] executes secure transactions on FPGAs. Other attacks [84], [85] have proven that TEEs are vulnerable to access pattern leakage. This resulted in works that combine TEEs and *oblivious data processing*, often ORAM-based [25], [26]: Opaque [18] supports distributed analytics, ObliDB [19] is a database engine, Obladi [63] introduces oblivious transactions, and Graphos [20] implements oblivious graph algorithms. However, none have considered oblivious stream processing.

Other works have studied **oblivious relational (static) joins**. Chang et al. [28] introduced an index NLJ for trusted proxy without support for ad-hoc updates, a central component of stream joins. Our OCA algorithms address this limitation through OAPPEND, enabling incremental updates without full reconstruction. Krastnikov et al. [41] proposed the first non-quadratic join for non-FK setups, also with no support for dynamic workloads. ObliDB [19] included a partially-oblivious (i.e., assuming oblivious memory) hash join using TEEs. Opaque [18] presented a simple and efficient FK join using a single oblivious sort and a sequential scan. Our joins improve [18] by replacing the oblivious sort with a more efficient OAPPEND. Finally, Mavrogiannakis et al. [42] scale oblivious joins to parallel environments. Yet, we are the first to study encrypted stream joins in the trusted hardware model.

Stream Joins is a well-studied topic in the database community. Kang et al. [53] introduced the three steps of a sliding window join. Joins were proposed for parallel [86] and distributed [87] execution, FPGAs [88], approximate joins [89], [90]. In addition, industry has proposed systems like Apache Flink [7] and Spark Structured Streaming [45]. While other systems have addressed geo-distribution [91] and adaptive stream synchronizations [92] , none consider execution in untrusted environments.

Others have worked on **privacy in dynamic environments**. Whisper [93] computes aggregations in the MPC model (similar to [94], [95]). Ostrovsky et al. [96] and Bethencourt et al. [97] investigated private stream search. In summary, existing privacy-preserving stream systems operate in different threat models (i.e., DP - targeting public datasets, and MPC - assuming non-collusion) and focus only on aggregations, a simpler database operator. This renders them indirect competitors. Recall that obliviousness protects the computation but not its result. While out of scope, Differential Privacy (DP) [98] and padding [34] can complement obliviousness, reducing query result leakage [99]–[102].

III. OBLIVIOUS STREAM JOIN PROCESSING

We present the first formal security scheme for private stream joins (§ III-A) and define its real/ideal security game (§ III-B), and formally define the problem (§ III-C).

A. Oblivious Stream Join Definitions

We introduce an oblivious stream join (OSJ) scheme. OSJ is a one-party protocol and consists of two algorithms: OSJ-INIT and OSJ-JOIN. The former initializes the server's state, and the latter performs a join operation:

- $EM \leftarrow$ OSJ-INIT$(1^\lambda, \mathcal{D})$: takes as input a security parameter λ and the data collection $\mathcal{D}$. It outputs the server's encrypted state EM, storing the stream windows content.
- $(\mathbf{T}, EM) \leftarrow$ OSJ-JOIN$(\mathcal{B}, EM)$: the client's input is a batch $\mathcal{B}$ of any number of tuples. The output consists of the join result $\mathbf{T}$ and the updated encrypted state EM.

Algorithm 1 OSJ real-ideal security experiments.

$bit \leftarrow \mathbf{Real}^{OSJ}_{\Pi,Adv}(\lambda)$:

1: $(st_A, \mathcal{D}) \leftarrow Adv(1^\lambda)$; $EM \leftarrow$ OSJ-INIT$(1^\lambda, \mathcal{D})$
2: **for each** $k = 1$ *to* q **do** ▷ q: polynomial #batches
3: $(\mathcal{B}_k, st_A) \leftarrow Adv(EM, \mathbf{T}_1, ..., \mathbf{T}_{k-1}, st_A)$
4: $(\mathbf{T}_k, EM) \leftarrow$ OSJ-JOIN$(\mathcal{B}_k, EM)$
return $bit \leftarrow Adv(\mathbf{T}_1, ..., \mathbf{T}_k, st_A)$

$bit \leftarrow \mathbf{Ideal}^{OSJ}_{Sim,Adv,\mathcal{L}}(\lambda)$:

1: $(st_A, \mathcal{D}) \leftarrow Adv(1^\lambda)$; $EM \leftarrow$ IDEALOSJ-INIT$(1^\lambda, \mathcal{D})$
2: **for each** $k = 1$ *to* q **do** ▷ q: polynomial #batches
3: $(\mathcal{B}_k, st_A) \leftarrow Adv(EM, \mathbf{T}_1, ..., \mathbf{T}_{k-1}, st_A)$
4: $(\mathbf{T}_k, EM) \leftarrow$ IDEALOSJ-JOIN$(\mathcal{L}(\mathcal{B}_k), st_A)$
return $bit \leftarrow Adv(\mathbf{T}_1, ..., \mathbf{T}_k, st_A)$

B. Security Game

OSJ's security proof is based on access simulation constructed using public information (as in standard ORAM security [25]). We define a real/ideal security game, where the attacker interacts with a machine running the real OSJ protocol ("real") or a machine running an OSJ simulator ("ideal"). The simulator utilizes only publicly available information obtained from the execution trace defined by a leakage function. Intuitively, our OSJ construction is secure if the attacker has no advantage in guessing which world ("real" or "ideal") it operates in.

Definition 1. *Suppose (*OSJ-INIT, OSJ-JOIN*) is an OSJ scheme based on the above definition, let $\lambda \in \mathbb{N}$ be the security parameter, and consider experiments* ***Real****(λ) and* ***Ideal****$_\mathcal{L}(\lambda)$ presented in Algorithm 1, where $\mathcal{L}$ is a leakage function. OSJ is $\mathcal{L}$-secure if for all polynomial-size adversaries* `Adv`*, there exist polynomial-time simulators* IDEALOSJ-INIT *and* IDEALOSJ-JOIN*, such that:*

$\left|\Pr[\mathbf{Real}^{OSJ}_{\Pi,Adv}(\lambda) = 1] - \Pr[\mathbf{Ideal}^{OSJ}_{Sim,Adv,\mathcal{L}}(\lambda) = 1]\right| \leq \mathsf{negl}(\lambda)$

C. Problem Definition

We perform a binary equi-join over two encrypted streams, $\mathcal{S}_R$ and $\mathcal{S}_S$, containing tuples $< \tau, k, p >$, with timestamp, key, and payload, respectively. We define two count-based, sliding windows, $\mathcal{W}_R$ and $\mathcal{W}_S$, of sizes w_R and w_S, and a *leakage function* $\mathcal{L}$ that defines what a secure protocol is allowed to leak. Given $\mathcal{W}_R$, $\mathcal{W}_S$, and $\mathcal{L}$, an OSJ algorithm appends tuple pairs (r, s) exactly once to an append-only join result set iff the tuples satisfy the join predicate P and at the time when r arrives, s is in $\mathcal{W}_S$ (similarly, when s arrives, r is in $\mathcal{W}_R$). In addition, the algorithm is $\mathcal{L}$-secure according to Def. 1. Formally: $\mathcal{S}_R \bowtie_{\mathcal{L}} \mathcal{S}_S = \{(r, s) | P(r, s), r \in \mathcal{W}_R, s \in \mathcal{W}_S\}$

Our definition aligns with foundational streaming research [53], [86], [88] and standard SPSs [7], [8], [45], [92].

Limitations. We process streams in the order of arrival. Our solutions are generalizable to other data-independent window types (e.g., session or tumbling windows). We leave out for future work windows with conditional logic (e.g., threshold windows), where branching leaks access patterns.

IV. Leakage Taxonomy

Privacy-preserving stream processing is fundamentally challenging because it alters the assumptions of today's secure computation. Oblivious joins now operate over *unbounded inputs*, under *strict latency constraints*, and with *evolving state*, rendering traditional static security models inadequate. Moreover, the lack of formal definitions for oblivious stream joins hinders fair comparison between solutions that assume different threat models. This creates a clear need for a framework to reason about privacy guarantees in streaming contexts.

We address this gap by introducing a leakage taxonomy framework for privacy-preserving stream joins. At its core, the framework formalizes five leakage functions that capture adversarial observability over time, allowing to precisely describe what an attacker infers in different scenarios. We define these functions in terms of a dynamic bipartite graph [103], referred to as the *join graph* [17], which evolves as new data arrive and join matches are recomputed. While our framework establishes foundational primitives for oblivious binary stream joins, it naturally extends to multi-way joins and other downstream operators with similarly defined leakage functions. We defer these extensions to future work.

Formally, graph $G(t) = (V_R(t), V_S(t), E(t))$ is a join graph at time t, where V_R and V_S are vertex sets of encrypted tuples stored in windows $\mathcal{W}_R$ and $\mathcal{W}_S$ of unbounded streams $\mathcal{S}_R$ and $\mathcal{S}_S$, and E is the set of edges denoting join matches between them. Each edge $e \in E$ connects a tuple between V_R and V_S. We denote edges incident to a vertex v as $E_v(t)$, the degree of v as $deg_v(t)$, and vertices connected to (i.e., matched tuples) v as $J_v(t)$. Tuples arrive in batches $\mathcal{B}$, as in many SPSs [6].

We propose five leakage functions ($\mathcal{L}_0$-$\mathcal{L}_4$), capturing progressively stronger privacy guarantees. They range from a leakage equivalent to deterministic (DET) encryption revealing complete join graph, to a leakage with full obliviousness leaking only public parameters. Figure 1 visualizes the intuition behind $\mathcal{L}_0$-$\mathcal{L}_4$. At a high level, $\mathcal{L}_0$ and $\mathcal{L}_1$ protect only data content via weak and strong data encryption, while exposing access patterns. $\mathcal{L}_2$ hides tuple-level access patterns but leaks volume information, while $\mathcal{L}_3$ generalizes this to batch-level leakage. Finally, $\mathcal{L}_4$ hides all access patterns, achieving maximal privacy. $\mathcal{L}_0$-$\mathcal{L}_2$ execute per-tuple, producing one output tuple per match, while $\mathcal{L}_3$-$\mathcal{L}_4$ execute per batch, outputting the batch join result ($\mathcal{L}_3$) or batch Cartesian product ($\mathcal{L}_4$).

The existing access pattern attacks [11]–[13], [21]–[23], [50] have proven $\mathcal{L}_0$/$\mathcal{L}_1$-secure static solutions insufficient, as they leak enough to recover plaintext results. On the contrary, while $\mathcal{L}_4$ offers the strongest protection, its worst-case complexity is often prohibitive. However, FK join constraints allow for bounded output cardinality, making $\mathcal{L}_4$-secure FK joins practical. In subsequent sections, we demonstrate that, through careful design, efficient $\mathcal{L}_4$-secure FK joins can be achieved.

Table I classifies prior secure static joins within this taxonomy as originally presented (e.g., with/out padding). While we include MPC and *trusted proxy* works, it is not adequate to directly compare their performance due to fundamental differences in communication (multi- vs. single-party), cryptographic primitives (secret sharing vs. TEE isolation), and security assumptions (non-collusion vs. hardware trust). We now define the leakage functions in terms of graph $G(t)$.

$\mathcal{L}_0$: Deterministic Confidentiality

Hint: Leak the full join graph and its history.

For each batch $\mathcal{B}$ between t_0 and t_i, the adversary learns the edges $E(t)$ (exact join matches) and, for each vertex $b \in \mathcal{B}$, its degree $deg_b(t)$ (i.e., volume pattern) and join indices $J_b(t)$. The adversary can correlate current data with expired tuples (e.g., dashed line in Figure 1a) and reconstruct historical join results. For instance, the DET-encrypted equi-joins in Always Encrypted [82] and CryptDB [81] are $\mathcal{L}_0$-secure. Formally: $\mathcal{L}_0(G(t_i)) = \left\langle (E(t_j), (deg_b(t_j), J_b(t_j))_{b \in \mathcal{B}_j})_{j \in \{0..i\}} \right\rangle$

$\mathcal{L}_1$: Randomized Confidentiality

Hint: Leak the current join graph without historical links.

$\mathcal{L}_1$ improves upon $\mathcal{L}_0$ by removing the adversary's ability to link current tuples to historical data. It reveals only the current join graph (i.e., results and degrees from the latest batch), but not if tuples match previously seen data. This is equivalent to RND encryption within a TEE, where each batch is encrypted independently. Figure 1b illustrates that $\mathcal{L}_1$ protects links to past data (i.e., the absence of a green dashed line in $T + 2$). Effectively, the adversary has no benefit in storing historical data. Solutions based on TEEs with RND encryption, such as Intel SGXv1, are $\mathcal{L}_1$-secure, for example, EnclaveDB [58] and CrkJ [104]. Formally: $\mathcal{L}_1(G(t_i)) = \left\langle (E(t_j), (deg_b(t_j), J_b(t_j))_{b \in \mathcal{B}_j})_{j \in \{i-1..i\}} \right\rangle$

$\mathcal{L}_2$: Oblivious Lookups

Hint: For every tuple lookup, leak only the volume pattern.

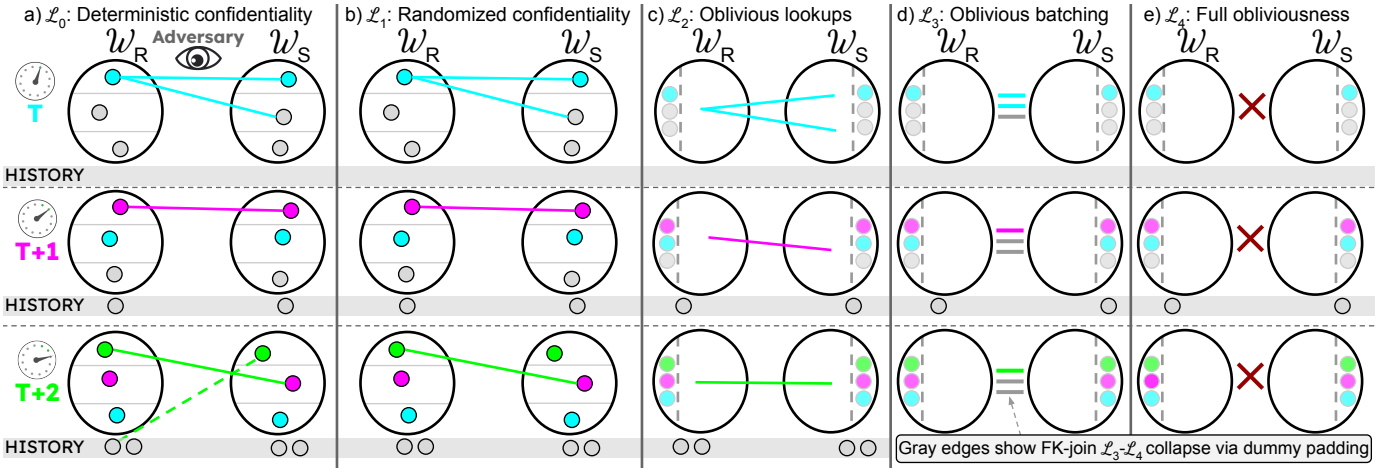

Fig. 1: Stream join graph elements visible to adversaries over time per leakage profile. Colors indicate time spans between batch additions, dashed lines show indirectly learned matches, and gray edges represent optional dummy tuples leading to $\mathcal{L}_4$.

TABLE I: Examples and classification of related secure static join work in their default mode, and the complexity per tuple of all our proposed stream joins. We denote window size as N, batch size as m, and batch output size as T.

	Non-FK Joins			FK Joins		
LF	**Static join prior work**	**Our stream join**	**Complexity**	**Static join prior work**	**Our stream join**	**Complexity**
$\mathcal{L}_0$	[81], [82]	*SHJ-L0*	$O(\log N)$	-	-	-
$\mathcal{L}_1$	[58]	*SHJ-L1*	$O(\log N)$	[104]	-	-
$\mathcal{L}_2$	[28]	*SHJ-L2*	$O(\log^2 N)$	[19] (no padding)	*FK-EPHI-L2*	$O((N \log N)/m)$
$\mathcal{L}_3$	[105], [17], [41], [102]	*SHJ-L3* *NFK-JOIN-L3*	$O(T \log^2 N)$ $O((N \log^2 N)/m)$	[18] (no padding)	*FK-MERG-L3* *FK-SORT-L3*	$O((N \log N)/m)$ $O((N \log^2 N)/m)$
$\mathcal{L}_4$	[55], [56], [64], [106]	*SHJ-L4*	$O(N \log^2 N)$	[18], [19]	*FK-MERG-L4* *FK-SORT-L4*	$O((N \log N)/m)$ $O((N \log^2 N)/m)$

$\mathcal{L}_2$ introduces oblivious data access, leaking only the number of join matches (i.e., tuple degree) per tuple at insertion time, without revealing which tuples matched. Match identities, join indices, and correlations across batches remain private. Figure 1c visualizes it as edges not incident to any vertex. This leakage profile commonly arises in ORAM-based systems [19], [20], [27], [28], [36], [37]. Formally: $\mathcal{L}_2(G(t_i)) = \langle |E(t_i)|, (deg_b(t_i))_{b \in \mathcal{B}_i} \rangle$

$\mathcal{L}_3$: Oblivious Batching

Hint: Leak intermediate output cardinality per micro-batch.

$\mathcal{L}_3$ further reduces leakage by revealing only the total number of join results per batch, instead of per tuple. While individual tuple contributions remain hidden, the aggregate output volume is exposed to the adversary. Note that $\mathcal{L}_3$ is easily tunable: for a single-tuple batch, it becomes $\mathcal{L}_2$, while when padding the result to worst-case cardinality, it achieves full obliviousness. This profile is typically assumed in static oblivious joins [41], [42]. Formally: $\mathcal{L}_3(G(t_i)) = \langle |E(t_i)|, deg_{\mathcal{B}_i}(t_i) \rangle$,
where $deg_{\mathcal{B}_i}(t_i) = \sum_{b \in \mathcal{B}_i} deg_b(t_i)$.

$\mathcal{L}_4$: Full Obliviousness

Hint: Leak no information beyond public metadata.

$\mathcal{L}_4$ represents the strongest privacy with full data and access patterns obliviousness. The adversary learns nothing about the join graph beyond public parameters. Achieving $\mathcal{L}_4$ requires worst-case padding and deterministic, data-independent execution. Existing instances of $\mathcal{L}_4$ include Secrecy's nested-loop join [56] and Opaque's FK join [18]. Formally: $\mathcal{L}_4(G(t_i)) = \emptyset$

V. ORAM-Based Symmetric Hash Joins

Modern SPSs universally adopt SHJ [6]–[8], making it the natural starting point for privacy-preserving equivalents. Our design goal is to enhance the existing stream join approach with obliviousnes. We contribute five SHJ-style algorithms: $\mathcal{L}_0$/$\mathcal{L}_1$-secure algorithms (§V-A), followed by ORAM-based $\mathcal{L}_2$-secure (§V-B) and $\mathcal{L}_3$/$\mathcal{L}_4$-secure joins (§V-C).

A. $\mathcal{L}_0$- and $\mathcal{L}_1$-Secure Symmetric Hash Joins

$\mathcal{L}_0$/$\mathcal{L}_1$-secure joins are widely adopted in commercial [107], [108] and academic systems [109]–[111], e.g., CryptDB [81] ($\mathcal{L}_0$) and Always Encrypted [82] ($\mathcal{L}_1$). Despite numerous attacks proving them insecure [11]–[13], [21]–[23], [50], cloud providers still offer them in their portfolios [112].

We implement *SHJ-L0* and *SHJ-L1*, two stream joins that match the guarantees of existing systems and establish performance baselines akin to current industry practices. While identical in the high-level structure, they differ in implementation and deployment. Both algorithms follow the canonical SHJ procedure (see § II-A) for unbounded streams [53]. They: (i) search the opposite stream's window for matching tuples, (ii) insert the tuple into its window, and (iii) expire old tuples.

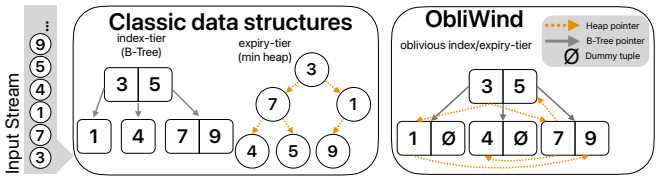

Fig. 2: Classic data structures (left) built from an input stream and OBLIWIND (right) that obliviously combines both.

Since we introduce no significant changes to the original idea, we defer the pseudocode to the full version of the paper.

We represent windows as maps (for fast lookups) and min-heaps (for expiration ordering). Specifically, window search performs a map lookup, insertion updates both the map and heap, and expiration executes heap extraction and map deletion. *SHJ-L0* processes DET-encrypted tuples outside trusted hardware. The streams are encrypted with AES-256 with a fixed initialization vector, allowing equality checks. In contrast, *SHJ-L1* processes RND-encrypted tuples inside a TEE, eliminating historical correlations.

Security. *SHJ-L0* enables observing the full join graph and correlating current with expired tuples, consistent with $\mathcal{L}_0$. *SHJ-L1* leaks the current join graph and prevents correlating it with historical data, matching $\mathcal{L}_1$ leakage. All formal proofs are provided in [113].

Performance. The dominant cost arises from heap extraction ($O(logN)$ time per tuple for N-size window). Consequently, both joins achieve $O(logN)$ complexity per tuple.

B. $\mathcal{L}_2$-Secure Symmetric Hash Join

$\mathcal{L}_2$-security hides access patterns for individual tuples. To achieve this, we must go beyond data encryption and protect data structures that manage the stream windows. Previous work addresses a different threat model and requires full rebuilding per batch [28]. This motivates a tuple-oblivious variant of SHJ, which we refer to as *SHJ-L2*. *SHJ-L2* follows the SHJ structure (as in the previous section), but replaces classic data structures (maps and heaps) with their oblivious, ORAM-based equivalents. This modification, although conceptually minor, substantially changes the security profile: instead of leaking full access patterns ($\mathcal{L}_0$-$\mathcal{L}_1$), *SHJ-L2* leaks only the degree of a tuple at insertion time, aligning with $\mathcal{L}_2$.

Prior oblivious maps [20], [36], [37] lack features required by sliding windows: efficient deletions and timestamp ordering. First, tuple deletion in existing ORAM-based maps is either prohibitively expensive [37] or not supported [20], [36]. Second, tuples expire based on arrival order (e.g., via timestamps), which is not natively supported by existing maps.

To address these challenges, we introduce OBLIWIND, a novel doubly-oblivious data structure for sliding stream windows. OBLIWIND (Figure 2) is a composite, two-tier data structure; It consists of an *index tier* for lookups and duplicate placement, and an *expiry tier* for time-ordered eviction. The key idea of the index tier is an oblivious B-Tree [24] atop Path-DORAM (similar to AVL tree in [36]). On the other hand, the expiry tier is a doubly-oblivious min-heap. This design enables OBLIWIND to perform all sliding window operations (i.e., search, insert, and expire) obliviously and supports efficient deletions through B-Tree's improved amortized performance (over AVL trees [20], [36], [37] with higher rebalancing costs). While join keys can have duplicates (e.g., many orders match a single customer), we need to be able to uniquely identify any tuple so that it can expire at the right time. Therefore, a tuple $< \tau, k, p >$ (timestamp, key, payload, respectively) is stored under a composite key $< k, \tau >$. In addition, by placing the timestamp second in the composite key, we ensure that tuples stored in a sorted tree (i.e., B-Tree) are sorted by key and timestamp. Thus, OBLIWIND addresses all the needs of oblivious stream windows , enabling ad-hoc updates without full rebuilt per batch (as in [28]).

We implement the *index tier* as a doubly-oblivious B-Tree [24] atop Path-ORAM [76] to support index operations. We ensure all operations are oblivious. Similar to [36], we extend the B-Tree with order-statistic tree [114] techniques to obliviously count duplicate elements, augmenting nodes with their subtree size. We modify insertion and deletion such that the size information is consistent across operations. These modifications are minor and do not affect OBLIWIND's security. Similarly, we use a doubly-oblivious version of Path Oblivious Heap (POH) [115] for the *expiry tier*. POH supports all classic heap operations. We order the elements in POH by timestamp, enabling efficient and private search of the oldest element. Below, we describe all operations in OBLIWIND.

- $search(k, \tau)$: finds an element with key k and timestamp τ. It executes one oblivious index read. If $\tau = 0$, it returns the oldest element with k (i.e., with the smallest timestamp).
- $size(k)$: returns the number of elements with key k. We run two oblivious index reads: with k and $\tau = 0$ (finds the oldest element with k), and with $k + 1$ and $\tau = 0$ (finds the oldest element with *next* key greater than k). The difference in their indices is the number of elements with k.
- $insert(\tau, k, p)$: inserts a new element with a composite key $< k, \tau >$ into the index and heap. B-Tree insertion ensures the tree's correctness by adjusting the size information in visited nodes and obliviously rebalancing the tree.
- $expire()$: extracts the minimum element using heap pointers and deletes it from the index. Deletion readjusts the size information across nodes and rebalances the B-Tree.

We implement *SHJ-L2* inside an Intel SGXv2 enclave with OBLIWIND stream windows. Notably, OBLIWIND integrates an in-enclave ORAM server. This change removes the need for TEE-client-server encryption, as all data is now stored in EPC. Additionally, all operations perform fully within the enclave, avoiding costly transitions. Although this increases the TCB, in-enclave components are a standard practice in secure systems [18], [58], [82] and have no security implications. Compared to OMIX++ [20], a recent AVL-based oblivious map, OBLIWIND improves search performance by 51%, while significantly expanding functionality.

Security. Access patterns in OBLIWIND are protected

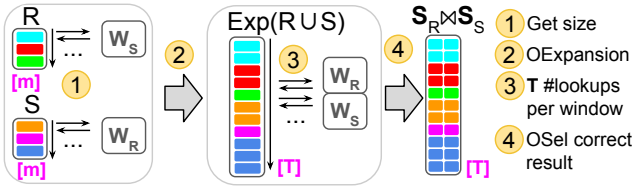

Fig. 3: *SHJ-L3* obliviously multiplies index queries to achieve $\mathcal{L}_3$. Collection sizes (in purple) are publicly known.

through a doubly-oblivious design. *SHJ-L2* leaks the tuple's degree at insertion time (i.e., how many matches it produces), which aligns with $\mathcal{L}_2$.

Performance. OBLIWIND operations have $O(\log N)$ cost. This compounds to $O(\log^2 N)$ per tuple when ORAM-stored.

Algorithm 2 Pseudocode of *SHJ-L3* and *SHJ-L4*.

Input: batches R and S of m tuples per stream
Output: join results without/with padding

1: **procedure** JOIN(tuples R, tuples S):
2: $T = 0; sizes \leftarrow \emptyset$
3: $\mathcal{W}_R.insert(R)$
4: **for** $i \in [0, m-1]$ **do**
5: $sizes[i] \leftarrow \mathcal{W}_S.size(R[i])$
6: $T \mathrel{+}= sizes[i]$
7: **for** $i \in [0, m-1]$ **do**
8: $sizes[i+m] \leftarrow \mathcal{W}_R.size(S[i])$
9: $T \mathrel{+}= sizes[i+m]$
10: WORSTCASEPADDING$(sizes, T)$
11: $res.1 \leftarrow$ OEXPANSION$(R \cup S, sizes)$
12: **for** $i \in [0..T-1]$ **do**
13: $p \leftarrow res[i].1$
14: $p_1 \leftarrow \mathcal{W}_R.search(p)$
15: $p_2 \leftarrow \mathcal{W}_S.search(p)$
16: $res[i].2 \leftarrow$ OSEL$(p.tid, p_1, p_2)$
17: $\mathcal{W}_S.insert(S)$
18: $\mathcal{W}_R.expire(); \mathcal{W}_S.expire()$
19: **return** res

C. $\mathcal{L}_3$- and $\mathcal{L}_4$-Secure Symmetric Hash Joins

While ORAM indices enabled tuple-level obliviousness, point-query design limits them in batch processing. This raises a question: how to leverage index-based joins and achieve batch-level ($\mathcal{L}_3$) or full ($\mathcal{L}_4$) obliviousness without revealing intermediate results and without new ORAM constructions?

We propose a strategy to hide tuple-level volume pattern of point queries. Instead of retrieving all join matches in a single query revealing the per-tuple degree, we perform a publicly-known number of single-tuple index queries, matching the final join output size. This hides the contributions of a single tuple, thereby enhancing privacy of existing solutions [28].

The core idea is a three-phase procedure (Figure 3). First, we compute the join cardinality per tuple in the input batches R and S. Specifically, we query each tuple's opposite window for its key size (1). We aggregate these values into a public total T, determining the number of index lookups. Second, we obliviously expand the input batches into an array of length T (2). Tuples are duplicated according to their join size (e.g., a tuple with three matches appears three times). This well-studied oblivious operator [41], [42], denoted OEXPANSION, prevents from learning individual tuple degrees. Finally, for each tuple in the expanded array, we perform one lookup in both windows (3). The correct join match is selected using OSEL [20], which obliviously selects the correct tuple and emits the result (4). After execution, we insert the batches into the windows and expire old tuples. Similar to the previous section, we represent windows with OBLIWIND.

Algorithm 2 shows the pseudocode of *SHJ-L3*. Its novelty sits in hiding the intermediate results of index queries. First, we determine the output cardinality of each tuple (lines 4-9). Then, we expand the input and query both windows T times while obliviously selecting the join results (lines 11-16). Finally, we retire tuples using already known procedures.

To achieve $\mathcal{L}_4$, we add worst-case padding. In line 10, we pad the total number of window lookups T to the maximum join output size (i.e., the window size per input tuple). This ensures that no information is leaked, including the total join cardinality. We call the resulting algorithm *SHJ-L4*.

Security. OBLIWIND supports all doubly-oblivious window operations. *SHJ-L3* leaks the join result size per batch, while hiding per-tuple access patterns, conforming with $\mathcal{L}_3$. *SHJ-L4* hides the output size via worst-case padding, achieving $\mathcal{L}_4$.

Performance. The dominant cost comes from OBLIWIND's operations incurring $O(\log^2 N)$. For output size T, *SHJ-L3* completes in $O(T \log^2 N)$, and *SHJ-L4* in $O(N \log^2 N)$.

VI. OBLIVIOUS COMPUTATION APPROACH JOINS

Previosuly, we showed that achieving all leakage profiles via oblivious indices is possible. However, these algorithms carried a heavy performance baggage - the underlying ORAM. Even the most efficient enclave ORAMs [20], [37] are unsuitable for streaming due to high latency and low scalability in terms of collection size. Moreover, while fully oblivious non-FK joins pad to a prohibitive quadratic size, worst-case padding becomes acceptable in FK joins (Figure 1d-e), where the complexity is substantially reduced. In this section, we propose non-ORAM FK joins that exploit the semantical assumptions, as well as their extensions to non-FK schemas. We propose Oblivious Computation Approach (OCA) $\mathcal{L}_3$- and $\mathcal{L}_4$-secure algorithms (§ VI-A), followed by a $\mathcal{L}_2$-secure join (§ VI-B), and extensions to non-FK stream joins (§ VI-C).

A. $\mathcal{L}_3$- and $\mathcal{L}_4$-Secure OCA FK Joins

Existing oblivious static joins rely on (multiple) oblivious sorts of the entire dataset [17], [18], [41] or full rebuilding [28]. Despite recent advancements [71], [116], oblivious sorting remains a performance barrier. Moreover, in streaming, windows are *almost* sorted after the previous batch. Thus, to achieve performant streaming, we must avoid full sorts.

Implementing windows as sorted collections avoids the costly sorting process. In a FK join, sorted windows allow us to proceed directly to the linear scan of the windows and join the primary key with foreign key records (similar to [18]). Therefore, the challenge is how to efficiently and obliviously maintain a sorted collection in a micro-batch scenario.

Algorithm 3 OAPPEND operation.

Input: m-tuple batch T, sorted collection C of size n
Output: sorted collection of $(n+m)$ tuples

1: **procedure** OAPPEND(tuples T, array C):
2: OSORT(T)
3: $T_{exp} \leftarrow$ PADD(T, n)
4: $C \leftarrow$ OMERGE(C, T_{exp}) ▷ $O(2nlog2n)$
5: **return** $C[0..m+n)$

We propose OAPPEND (Algorithm 3) that obliviously appends a batch to a sorted collection while preserving its order. Our goal is to reduce the complexity of a full oblivious sort [18] and avoid full rebuilding (as in [28]). First, we obliviously sort the micro-batch; Its cost is negligible as the batch size is small (i.e., fits in L1 cache). Then, we pad it with dummy infinity tuples such that the batch and window are both sorted and of equal sizes. In this setup, we can utilize an oblivious merge [70], a simpler (i.e., $O(N \log N)$) primitive that merges two sorted sequences into a sorted collection. Therefore, we obliviously merge the batch with the window and trim the dummy records.

Using OAPPEND, we now maintain windows as sorted collections. With that, we propose *FK-MERG-L3* and *FK-MERG-L4*, $\mathcal{L}_3$-, and $\mathcal{L}_4$-secure FK stream join algorithms. The high-level intuition is to append the input batch to opposite stream's sorted window and obtain the join matches with a full sequential scan. With every scanned tuple, we emit a join match or a dummy tuple. Our algorithm improves Opaque's static join [18] complexity to $O((N \log N)/m)$, while adapting to unbounded environments. Note that the result is padded to the FK worst-case, achieving $\mathcal{L}_4$-security.

We further optimize the algorithms by running two smaller joins with worst-case padding. The idea is separately joining batches with opposite windows. Our join becomes: $((R \cup \mathcal{W}_R) \bowtie S) \cup (R \bowtie \mathcal{W}_S)$, where R is the PK-batch and S is the FK-batch. We benefit twofold from this optimization; We perform two smaller joins and achieve result uniqueness.

Figure 4 shows how we achieve worst-case padding and result uniqueness. Stage 1 obliviously appends R to $\mathcal{W}_R$. Next, we append R and S to their opposite windows, getting T_1 and T_2. Finally, we scan both collections and emit join matches. Intermediate sizes (purple text) are publicly known.

Algorithm 4 details *FK-MERG-L3* and *FK-MERG-L4*. We begin by appending batch R to its window (line 2). Next, we join batch S with $\mathcal{W}_R$ by merging them into T_1 (line 3) and performing a SCAN on T_1 (line 4). SCAN sequentially scans a collection, while tracking the last PK-tuple and joining it with the current FK-tuple (as in [18]). For each scanned tuple, it emits a join match or a dummy tuple. Then, we perform the

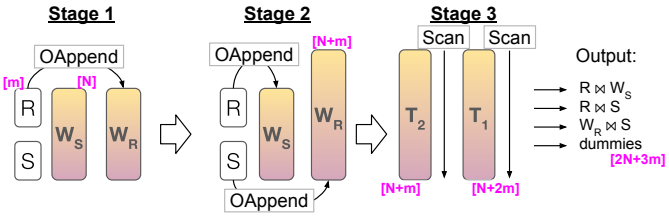

Fig. 4: *FK-MERG-L4* obliviously joins unsorted (white) batches with sorted (colored) windows. Purple text indicates size of a collection.

second of the smaller joins in lines 5-6. Finally, we append S to its window (line 7) and retire old tuples from both windows. We implement ORETIRE to expire tuples from windows by: (i) marking old tuples via sequential scan based on the timestamp, (ii) obliviously compacting the windows on the marked bit, and (iii) trimming old tuples. Additionally, *FK-MERG-L3* compacts the result to remove dummy tuples (line 10). The order of operations maintains the result uniqueness.

Algorithm 4 Pseudocode of *FK-MERG-L3* and *FK-MERG-L4* stream joins.

Input: batch of m tuples per stream
Output: join results

1: **procedure** *FK-MERG-L4*(tuples R, tuples S):
2: $\mathcal{W}_R \leftarrow$ OAPPEND($R, \mathcal{W}_R$)
3: $T_1 \leftarrow$ OAPPEND($S, \mathcal{W}_R$)
4: $res =$ SCAN(T_1)
5: $T_2 =$ OAPPEND($R, \mathcal{W}_S$)
6: $res.append$(SCAN(T_2))
7: $\mathcal{W}_S \leftarrow$ OAPPEND($S, \mathcal{W}_S$)
8: ORETIRE($\mathcal{W}_R$)
9: ORETIRE($\mathcal{W}_S$)
10: OCOMPACTION(res)
11: **return** res

Security. Both algorithms use OAPPEND, oblivious merge, sort, and sequential scan, all proven secure. Two smaller joins leak no information thanks to worst-case padding. With compaction, the leakage includes total volume pattern $|E|$ and volume pattern per batch deg_B, matching $\mathcal{L}_3$. Without compaction, it leaks no information, which matches $\mathcal{L}_4$. Hence, *FK-MERG-L4* is $\mathcal{L}_4$-secure and *FK-MERG-L3* is $\mathcal{L}_3$-secure.
Performance. Sorting cost of an m-tuple batch is negligible. Other operations are $O(NlogN)$ for N-sized windows, resulting in $O(NlogN/m)$ complexity per tuple.

B. $\mathcal{L}_2$-Secure OCA FK Join

In oblivious computation, data is constantly relocated to avoid correlations across operations. Yet, a single access reveals no access patterns. Therefore, we can define ephemeral (i.e., one-time) data structures and access all their records exactly once without leaking any patterns. Further, Asharov et al. [71] observed that an oblivious shuffle breaks cross-execution correlations. This opens an exciting opportunity:

an oblivious shuffle and a non-oblivious procedure yields an oblivious algorithm (e.g., sorting [71]). These two observations, oblivious shuffling and rebuilding ephemeral structures, indicate a strong periodicity also seen in streaming: While the former provides a "fresh start", the latter allows for one-time obliviousness. This "access-once-before-reshuffling" property is similar to square-root and hierarchical ORAMs [25].

We propose *FK-EPHI-L2* ("foreign-key-ephemeral-index"), a micro-batch, $\mathcal{L}_2$-secure FK stream join algorithm. Its core idea is a three-step procedure. First, obliviously shuffle both windows with the incoming batches. Second, build an ephemeral, non-oblivious hash table on the FK-relation (i.e., with duplicates). Third, probe it with unique requests from the primary-key relation (i.e., without duplicates), collect the join matches, and destroy the index. The remaining part of the algorithm requires to deduplicating join matches and retiring old records obliviously .

Algorithm 5 shows the pseudocode of *FK-EPHI-L2*. We begin by appending batches to their windows and obliviously shuffling both collections (lines 2-5). Next, we build a non-oblivious hash table on $\mathcal{W}_S$ (line 6) and probe it with unique records from $\mathcal{W}_R$ (lines 8-9). We collect the matches in collection res. Similar to Apache Spark [45], we deduplicate join results (line 10). We implement OSELECTION for deduplication. It works similarly to ORETIRE (see § VI-A) using timestamps to mark new (i.e., to-be-emitted) records. Finally, *FK-EPHI-L2* retires old tuples using ORETIRE (lines 11-12).
Security. *FK-EPHI-L2* is oblivious, excluding index building and probing. However, index accesses are distinct, leaking only volume pattern per access. There are no repeated accesses that leak tuple correlations. Hence, we leak the degree of each tuple but no cross-batch or cross-tuple correlations, fitting $\mathcal{L}_2$.
Performance. Shuffle incurs the highest cost $O(N \log N/m)$ per tuple for N-size window and m-size batch.

Algorithm 5 Pseudocode of *FK-EPHI-L2* stream join.

Input: batch of m tuples per stream
Output: join results

```
1:  procedure FK-EPHI-L2(tuples R, tuples S):
2:      W_R.insert(R)
3:      W_S.insert(S)
4:      OSHUFFLE(W_R)
5:      OSHUFFLE(W_S)
6:      ht ← NONOBLIVIOUSHASHTABLESETUP(W_S)
7:      res ← ∅
8:      for t ∈ W_R do
9:          res.append(ht.search(t))
10:     res ← OSELECTION(res)
11:     ORETIRE(W_R)
12:     ORETIRE(W_S)
13:     return res
```

C. OCA Non-FK Joins

Non-FK oblivious joins are more complex as they lack cardinality bounds lower than the Cartesian product. Without ORAM, the strawman oblivious, generic stream join involves a full window join for per batch. Thus, we now face the problem of generic static oblivious join [17] executed periodically, requiring result uniqueness. This introduces a new challenge as the existing works [17], [28], [41] are incapable of tracking new results after an incremental change.

We propose *NFK-JOIN-L3*, the first non-ORAM doubly-oblivious generic stream join algorithm. The main idea is a full oblivious join between windows while tracking tuples from the most recent batch. The key insight is to prune the results and emit only new matches.

To achieve this, we adapted an oblivious static join to streaming. We mark tuples from the recent batch and append them to their respective windows. We then obliviously join the windows and, using the mark, determine emitting a join match; We only emit a match if it contains at least one marked tuple. Effectively, we still perform a full join on both windows, while trimming redundant tuples.

We base our implementation on the state-of-the-art oblivious static join introduced by Krastnikov et al. [41]. Precisely, we modify its group dimension calculation stage; After concatenating and sorting windows, we perform a downward scan and mark tuples from $\mathcal{W}_S$ that join with at least one new tuple from $\mathcal{W}_R$. Next, we do the opposite - we perform an upward scan and mark tuples from $\mathcal{W}_R$ that join with at least one new tuple from $\mathcal{W}_S$. We now proceed to the original dimension calculation but increase the group dimensions for tuples that have been marked in the previous scans (i.e., that potentially contribute a new result tuple). From this point, we continue with the remainder of the original algorithm. At the end of the join algorithm, we have collected the join matches where at least one of the tuples comes from the most recent batch.
Security. *NFK-JOIN-L3* is based on a $\mathcal{L}_3$-secure join [41]. The modification adds two full sequential scans with no leakage. All remaining operations are oblivious, matching $\mathcal{L}_3$.
Performance. *NFK-JOIN-L3* has $O(N \log^2 N/m)$ complexity per tuple, with oblivious sort being the dominating factor.

VII. EXPERIMENTAL EVALUATION

In the previous sections, we proved the security of our stream joins. We now evaluate their performance. First, we estimate the cost of each leakage profile (§ VII-B). Second, we compare the performance of FK joins (§ VII-C) and Non-FK joins (§ VII-D). For each experiment set, we manipulate the key stream parameters, i.e., batch size and window size.

A. Setup

Platform. We ran all experiments on a machine with 2 x Intel Xeon 4416+ CPUs, 128 GB EPC, and 188 GB RAM. It ran Ubuntu 22.04.3 LTS OS with SGX SDK 2.24.100 and gcc 11.4.0.. We implemented all algorithms in C++ as a standalone library that isolates oblivious operations, enabling tight security control and precise performance measurements without production overhead,e.g., when integrating into systems like Apache Flink [7]. We conducted the experiments using TeeBench [32] with stream query support.

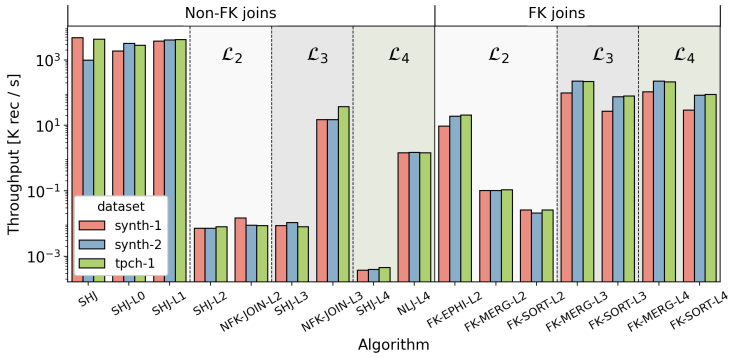

Fig. 5: The cost of oblivious stream joins.

TABLE II: Memory consumption and average latency of stream joins for dataset *synth-1*.

	Join	Algorithm	Peak memory usage [MB]	Latency [μs/tuple]
Non-Foreign Key		*SHJ*	28	4
		SHJ-L0	53	6
		SHJ-L1	29	3
		SHJ-L2	1798	$132 * 10^3$
		SHJ-L3	1798	$151 * 10^3$
		SHJ-L4	1798	$3554 * 10^3$
		NFK-JOIN-L2	30	$67 * 10^3$
		NFK-JOIN-L3	30	66
		NLJ-L4	16	613
Foreign Key		*FK-EPHI-L2*	33	226
		FK-MERG-L2	50	$9 * 10^3$
		FK-MERG-L3	50	18
		FK-MERG-L4	50	13
		FK-SORT-L2	48	$32 * 10^3$
		FK-SORT-L3	48	32
		FK-SORT-L4	48	29

Baselines. Despite a rich history of secure static joins, no direct competitors exist for oblivious stream joins. Our core baselines are *SHJ* (high performance, no privacy) and *NLJ-L4* (strong privacy, low performance), with *NLJ-L4* enabling indirect comparison to MPC nested-loop joins [54]–[56]. Direct comparisons with other MPC works [106] and *trusted proxy* [28] solutions are unfair due to differing communication models, cryptographic primitives, and security assumptions. We implemented a *SORT* family of oblivious stream FK joins that obliviously sort entire windows per join invocation and sequentially scan for join matches, resembling Opaque's static join [18]. *FK-SORT-L4* achieves $\mathcal{L}_4$-security, while *FK-SORT-L3* trims dummy tuples via oblivious compaction, achieving $\mathcal{L}_3$-security. Finally, we introduce two $\mathcal{L}_2$ baselines: *FK-MERG-L2* and *FK-SORT-L2*, single-tuple batch versions of their $\mathcal{L}_3$ equivalents. Both algorithms leak per-tuple degree, obtaining $\mathcal{L}_2$-security. Our main metric is the maximum sustainable throughput [30]. Since ORAM-based joins are not easily parallelizable, we report single-threaded performance. We abort experiments that exceed 6h.

Datasets. Oblivious computation performance depends solely on data size, not content, and is indistinguishable across inputs. Any same-size real and synthetic datasets perform identically [42]. Following related work [86], [88]–[90], [117], we use synthetic datasets: *synth-1* has two symmetric streams at 1000 [tuples/s], and *synth-2* has asymmetric streams at 1000 and 4000 [tuples/s]. As in [87], we use *tpch-1* dataset, joining TPC-H *customer* and *orders* tables on *c_custkey*. All tuples follow <*timestamp,key,payload*> format. We generated datasets with FK relationships for all FK joins [91]. Default parameters are 2^{16}-tuple windows and a 1-second batches (as in [45]). We use smaller windows that OCA joins support to enable comparison with ORAM-based joins. All runs are warmed up by filling the windows to avoid startup effects (i.e., windows not retiring old tuples). For results consistent across datasets, we report single-dataset numbers.

Implementation. *SHJ*, *SHJ-L0*, and *SHJ-L1* use bucket-chaining hash tables. *SHJ-L2*, *SHJ-L3*, and *SHJ-L4* use OBLIWIND. Additionally, we use expansion from [41]. The OCA joins employ modified oblivious sort, shuffle, and compaction from [116], OBLIVIOUSMERGE with bitonic merge [70], and standard library map for *FK-EPHI-L2*.

B. Privacy Cost in Oblivious Stream Joins

Given the absence of prior oblivious stream joins, a central question we address is: *What is the performance cost associated with each leakage level?* Based on related work [18], [20], [41], we expect ORAM-based approaches to underperform, whereas OCA joins will offer competitive throughput. To evaluate this, we benchmark the throughput of all proposed algorithms across three test datasets.

Figure 5 highlights three findings, which align with our intuition. First, basic encryption ($\mathcal{L}_0$/$\mathcal{L}_1$) incurs negligible overhead, with *SHJ-L0* and *SHJ-L1* matching native CPU performance. Second, OCA FK joins achieve high performance across leakages. e.g., *FK-MERG-L4* is only $4.3\times$ slower than *SHJ* on *synth-2*, while ensuring full confidentiality and obliviousness. Third, while ORAM-based joins (*SHJ-L2* to *SHJ-L4*) support generic joins, they are 3-6 orders of magnitude slower than OCA joins and 6-8 orders of magnitude slower than insecure *SHJ*, due to costly ORAM access. Thus, ORAM's generality comes at a prohibitive performance cost.

Next, we evaluated peak memory usage (via OS and *sgx-gdb*) and per-tuple latency. For tuple-at-a-time modes, latency reflects the time from join invocation to the emission of all matches. For micro-batch modes, it is the batch processing time divided by the batch size (assuming no batch fill delay). Table II summarizes the results; similar trends hold across datasets, which are omitted for brevity. Non-ORAM algorithms exhibit moderate memory usage, while ORAM-based joins consume nearly 2 GB even with 2^{16}-tuple windows, indicating poor scalability and posing a challenge for TEEs with limited secure memory (e.g., 8 GB in SGXv2 [118]). Latency reveals a similar pattern: *FK-MERG-L4* achieves $13\mu s$ per tuple, whereas oblivious SHJ joins $0.13 - 3.5s$ – too slow for real-time processing. Additionally, *FK-MERG-L2* and *FK-SORT-L2* perform poorly in tuple-at-a-time mode, reinforcing that these designs are better suited to micro-batching.

OCA algorithms outperform ORAM-based baselines by 3-6 orders of magnitude, while offering low latency.

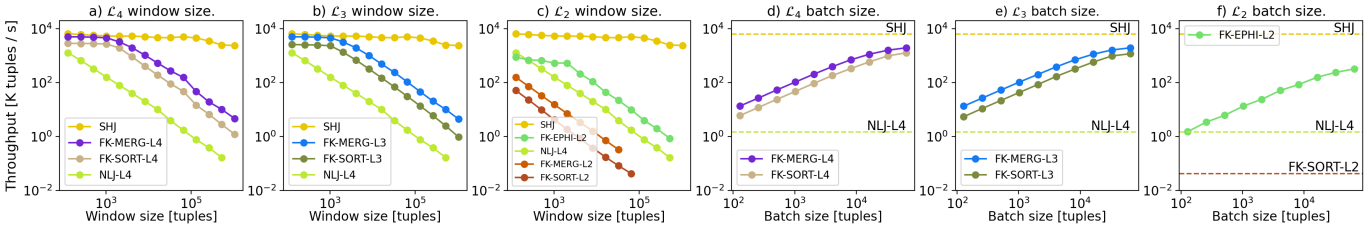

Fig. 6: Performance of FK stream join algorithms for $\mathcal{L}_2$-$\mathcal{L}_4$ leakages as a function of (a-c) window size and (d-f) batch size.

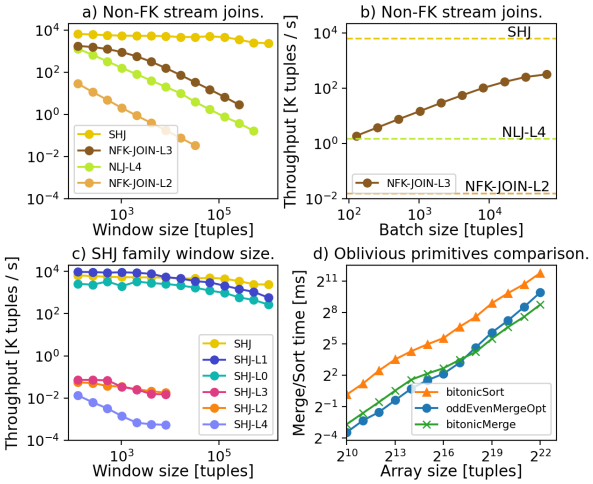

Fig. 7: (a-c) Performance of non-FK stream join algorithms. (d) Oblivious merge and sort performance.

C. Oblivious Foreign Key Stream Joins

We evaluate FK joins, a common streaming use case [91], with varying window and batch sizes. The *MERG* family consistently outperforms *SORT* across all leakage levels (Figures 6a-c), e.g., *FK-MERG-L4* achieves $4.6\times$ higher throughput than *FK-SORT-L4*, validating OAPPEND's efficiency. Micro-benchmarks (Figure 7d) confirm Odd-Even and Bitonic Merge outperform Bitonic Sort [70] by $> 2\times$. *FK-MERG-L4* outperforms *FK-MERG-L3* despite stronger privacy, due to reduced compaction overhead.

Among $\mathcal{L}_2$-joins, *FK-EPHI-L2* achieves the highest throughput but lags $\mathcal{L}_3$ joins by an order of magnitude due to expensive index construction, questioning $\mathcal{L}_2$'s justification. Finally, we examined the effect of larger batch size. Intuitively, it improves throughput as OAPPEND pads with fewer dummy tuples instead processing real ones. Yet, large batches can hurt responsiveness by increasing average latency. Figures 6d-f confirm that increasing batch size improves throughput across algorithms. Notably, *FK-MERG-L4* achieves $450\times$ higher throughput than *NLJ-L4* and is within $3.5\times$ of *SHJ*.

FK-MERG-L4 achieves the highest privacy and performance among FK joins with a $3.5\times$ slowdown compared to *SHJ*.

D. Oblivious Non-Foreign Key Stream Joins

We now measure the cost of going beyond FK constraints. Figures 7a-c report throughput across window and batch sizes for *SHJ* and *JOIN* non-FK families.

NFK-JOIN-L3 avoids ORAM's penalty, while *NFK-JOIN-L2* upholds previous $\mathcal{L}_2$ findings (Figure 7a). Comparing across families and join types, *NFK-JOIN-L3* performs 3-$10\times$ worse than *FK-MERG-L3*, underscoring FK efficiency gains. Even *NLJ-L4* outperforms oblivious *SHJ* joins, highlighting the severity of ORAM overhead. Lastly, *NFK-JOIN-L3* benefits from larger batches (Figure 7b), illustrating the latency-throughput tradeoff. Results for oblivious *SHJ* joins for large batch sizes are omitted due to infeasible runtimes.

Figure 7c illustrates that non-oblivious *SHJ-L0* and *SHJ-L1* maintain stable performance due to constant-time index operations, closely matching insecure *SHJ* with minimal encryption overhead. In contrast, oblivious variants (*SHJ-L2/L3/L4*) degrade severely as the window size increases due to expensive ORAM access. Recall that the SHJ family does not aim at improving ORAM's performance. Despite OBLIWIND's optimizations, ORAM-based joins remain unsuitable for latency-sensitive workloads. Additionally, the gap between *SHJ-L3* and *SHJ-L4* shows the impact of worst-case padding. Overall, ORAM's performance penalty renders it impractical.

NFK-JOIN-L3 pays an order of magnitude performance price for generic stream joins compared to FK-joins.

VIII. CONCLUSIONS

The existing SPSs lack privacy protection in untrusted environments and remain vulnerable to access pattern leakage despite using TEEs. We have introduced a formal security framework that defines five leakage functions, proposing new privacy-performance tradeoffs for stream join queries. We have contributed two families of stream join algorithms: an index-based (SHJ) approach and a computation-based approach (OCA). Our experiments demonstrate that OCA algorithms outperform ORAM-based joins by several orders of magnitude and operate within an order of magnitude of insecure baselines while offering strong privacy. The results demonstrate that privacy-preserving joins, particularly for foreign-key queries, are practical for real-world applications.

Future Work. While we presented principal oblivious stream join ideas, we left out many promising research directions. First, combining privacy techniques, such as obliviousness with differential privacy, would allow a gradual transition between leakage profiles. Second, our techniques may apply to other operators. The critical question is which components are reusable. Third, modern SPSs offer advanced features, such as fault tolerance. It is crucial to support them obliviously.

REFERENCES

[1] H. C. Ates, A. K. Yetisen, F. Güder, and C. Dincer, "Wearable devices for the detection of covid-19," *Nature Electronics*, vol. 4, no. 1, pp. 13–14, 2021.

[2] Meta, "The labirynth encrypted message storage protocol," Tech. Rep., 2023.

[3] Apple. (2024) Apple platform security. [Online]. Available: https://help.apple.com/pdf/security/en_US/apple-platform-security-guide.pdf

[4] WhatsApp. (2016) end-to-end encryption. [Online]. Available: https://blog.whatsapp.com/end-to-end-encryption

[5] TikTok. (2023) Petace in action: Enhancing privacy during friends matching in tiktok. [Online]. Available: https://developers.tiktok.com/blog/tiktok-practices-in-privacy-enhancing-technologies

[6] M. Zaharia, T. Das, H. Li, T. Hunter, S. Shenker, and I. Stoica, "Discretized streams: Fault-tolerant streaming computation at scale," in *Proceedings of the twenty-fourth ACM symposium on operating systems principles*, 2013.

[7] P. Carbone, A. Katsifodimos, S. Ewen, V. Markl, S. Haridi, and K. Tzoumas, "Apache flink: Stream and batch processing in a single engine," *The Bulletin of the Technical Committee on Data Engineering*, 2015.

[8] Apache. (2024) Apache storm. [Online]. Available: https://storm.apache.org/

[9] Netflix. (2016) Evolution of the netflix data pipeline. [Online]. Available: https://netflixtechblog.com/evolution-of-the-netflix-data-pipeline-da246ca36905

[10] D. Abadi, A. Ailamaki, D. Andersen, P. Bailis, M. Balazinska, P. Bernstein, P. Boncz, S. Chaudhuri, A. Cheung, A. Doan *et al.*, "The seattle report on database research," *ACM Sigmod Record*, vol. 48, no. 4, pp. 44–53, 2020.

[11] D. Cash, P. Grubbs, J. Perry, and T. Ristenpart, "Leakage-abuse attacks against searchable encryption," in *Proceedings of the 22nd ACM SIGSAC conference on computer and communications security*, 2015.

[12] G. Kellaris, G. Kollios, K. Nissim, and A. O'neill, "Generic attacks on secure outsourced databases," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, 2016.

[13] M. Naveed, S. Kamara, and C. V. Wright, "Inference attacks on property-preserving encrypted databases," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, 2015.

[14] S. Lee, M.-W. Shih, P. Gera, T. Kim, H. Kim, and M. Peinado, "Inferring fine-grained control flow inside {SGX} enclaves with branch shadowing," in *26th USENIX Security Symposium (USENIX Security 17)*, 2017, pp. 557–574.

[15] S. Fei, Z. Yan, W. Ding, and H. Xie, "Security vulnerabilities of sgx and countermeasures: A survey," *ACM Computing Surveys (CSUR)*, vol. 54, no. 6, pp. 1–36, 2021.

[16] J. Van Bulck, N. Weichbrodt, R. Kapitza, F. Piessens, and R. Strackx, "Telling your secrets without page faults: Stealthy page {Table-Based} attacks on enclaved execution," in *26th USENIX Security Symposium (USENIX Security 17)*, 2017, pp. 1041–1056.

[17] A. Arasu and R. Kaushik, "Oblivious query processing," in *Proc. 17th International Conference on Database Theory (ICDT), Athens, Greece, March 24-28, 2014*, 2014.

[18] W. Zheng, A. Dave, J. G. Beekman, R. A. Popa, J. E. Gonzalez, and I. Stoica, "Opaque: An oblivious and encrypted distributed analytics platform," in *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, 2017, pp. 283–298.

[19] S. Eskandarian and M. Zaharia, "Oblidb: oblivious query processing for secure databases," *Proceedings of the VLDB Endowment*, 2019.

[20] J. G. Chamani, I. Demertzis, D. Papadopoulos, C. Papamanthou, and R. Jalili, "Graphos: Towards oblivious graph processing," *Proceedings of the VLDB Endowment*, 2023.

[21] M. S. Islam, M. Kuzu, and M. Kantarcioglu, "Access pattern disclosure on searchable encryption: ramification, attack and mitigation." in *NDSS*, 2012.

[22] P. Grubbs, R. McPherson, M. Naveed, T. Ristenpart, and V. Shmatikov, "Breaking web applications built on top of encrypted data," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, 2016, pp. 1353–1364.

[23] E. M. Kornaropoulos, C. Papamanthou, and R. Tamassia, "The state of the uniform: Attacks on encrypted databases beyond the uniform query distribution," in *2020 IEEE Symposium on Security and Privacy (SP)*, 2020.

[24] X. S. Wang, K. Nayak, C. Liu, T. H. Chan, E. Shi, E. Stefanov, and Y. Huang, "Oblivious data structures," in *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*, 2014, pp. 215–226.

[25] O. Goldreich and R. Ostrovsky, "Software protection and simulation on oblivious rams," *Journal of the ACM (JACM)*, vol. 43, no. 3, pp. 431–473, 1996.

[26] R. Ostrovsky, "Efficient computation on oblivious rams," in *Proceedings of the twenty-second annual ACM symposium on Theory of computing*, 1990, pp. 514–523.

[27] G. Connell, "Technology deep dive: Building a faster oram layer for enclaves," 2012.

[28] Z. Chang, D. Xie, S. Wang, and F. Li, "Towards practical oblivious join," in *Proceedings of the 2022 International Conference on Management of Data*, 2022, pp. 803–817.

[29] Y. Li and M. Chen, "Privacy preserving joins," in *2008 IEEE 24th International Conference on Data Engineering*. IEEE, 2008, pp. 1352–1354.

[30] J. Karimov, T. Rabl, A. Katsifodimos, R. Samarev, H. Heiskanen, and V. Markl, "Benchmarking distributed stream data processing systems," in *2018 IEEE 34th international conference on data engineering (ICDE)*. IEEE, 2018, pp. 1507–1518.

[31] A. Spark. (2026) Streamingsymmetrichashjoinexec. [Online]. Available: https://github.com/apache/spark/tree/master/sql/core/src/main/scala/org/apache/spark/sql/

[32] K. Maliszewski, J.-A. Quiané-Ruiz, J. Traub, and V. Markl, "What is the price for joining securely? benchmarking equi-joins in trusted execution environments," *Proceedings of the VLDB Endowment*, 2021.

[33] S. Maiyya, S. C. Vemula, D. Agrawal, A. El Abbadi, and F. Kerschbaum, "Waffle: An online oblivious datastore for protecting data access patterns," *Proceedings of the ACM on Management of Data*, 2023.

[34] I. Demertzis, D. Papadopoulos, C. Papamanthou, and S. Shintre, "{SEAL}: Attack mitigation for encrypted databases via adjustable leakage," in *29th USENIX security symposium (USENIX Security 20)*, 2020, pp. 2433–2450.

[35] T. H. Chan, K. Chung, B. M. Maggs, and E. Shi, "Foundations of differentially oblivious algorithms," in *Proceedings of the Thirtieth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2019, San Diego, California, USA, January 6-9, 2019*, 2019.

[36] P. Mishra, R. Poddar, J. Chen, A. Chiesa, and R. A. Popa, "Oblix: An efficient oblivious search index," in *2018 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2018, pp. 279–296.

[37] A. Tinoco, S. Gao, and E. Shi, "{EnigMap}:{External-Memory} oblivious map for secure enclaves," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 4033–4050.

[38] A. Ahmad, K. Kim, M. I. Sarfaraz, and B. Lee, "OBLIVIATE: A data oblivious filesystem for intel SGX," in *25th Annual Network and Distributed System Security Symposium, NDSS 2018, San Diego, California, USA, February 18-21, 2018*, 2018.

[39] P. Williams, R. Sion, and A. Tomescu, "Privatefs: A parallel oblivious file system," in *Proceedings of the 2012 ACM conference on Computer and communications security*, 2012, pp. 977–988.

[40] S. Gueron, "A memory encryption engine suitable for general purpose processors," *Cryptology ePrint Archive*, 2016.

[41] S. Krastnikov, F. Kerschbaum, and D. Stebila, "Efficient oblivious database joins," *Proceedings of the VLDB Endowment*, 2020.

[42] A. Mavrogiannakis, X. Wang, I. Demertzis, D. Papadopoulos, and M. Garofalakis, "Obliviator: Oblivious parallel joins and other operators in shared memory environments," *Cryptology ePrint Archive*, 2025.

[43] Y. Wang, X. Zeng, S. Wang, and F. Li, "Jodes: Efficient oblivious join in the distributed setting," *arXiv preprint arXiv:2501.09334*, 2025.

[44] J. Verwiebe, P. M. Grulich, J. Traub, and V. Markl, "Survey of window types for aggregation in stream processing systems," *The VLDB Journal*, vol. 32, no. 5, pp. 985–1011, 2023.

[45] M. Armbrust, T. Das, J. Torres, B. Yavuz, S. Zhu, R. Xin, A. Ghodsi, I. Stoica, and M. Zaharia, "Structured streaming: A declarative api for real-time applications in apache spark," in *Proceedings of the 2018 International Conference on Management of Data*, 2018.

[46] Microsoft. (2024) Outputs from azure stream analytics. [Online]. Available: https://learn.microsoft.com/en-us/azure/stream-analytics/stream-analytics-define-outputs

[47] Google. (2024) Outputs from azure stream analytics. [Online]. Available: https://cloud.google.com/products/dataflow

[48] M. Idris, M. Ugarte, and S. Vansummeren, "The dynamic yannakakis algorithm: Compact and efficient query processing under updates," in *Proceedings of the 2017 ACM International Conference on Management of Data*, 2017, pp. 1259–1274.

[49] Q. Wang, X. Hu, B. Dai, and K. Yi, "Change propagation without joins," *Proceedings of the VLDB Endowment*, vol. 16, no. 5, pp. 1046–1058, 2023.

[50] P. Grubbs, M.-S. Lacharité, B. Minaud, and K. G. Paterson, "Learning to reconstruct: Statistical learning theory and encrypted database attacks," in *2019 IEEE Symposium on Security and Privacy (SP)*, 2019.

[51] R. Poddar, S. Wang, J. Lu, and R. A. Popa, "Practical volume-based attacks on encrypted databases," in *2020 IEEE European Symposium on Security and Privacy (EuroS&P)*, 2020.

[52] S. Schuh, X. Chen, and J. Dittrich, "An experimental comparison of thirteen relational equi-joins in main memory," in *Proceedings of the 2016 International Conference on Management of Data*, 2016, pp. 1961–1976.

[53] J. Kang, J. F. Naughton, and S. D. Viglas, "Evaluating window joins over unbounded streams," in *Proceedings 19th International Conference on Data Engineering (Cat. No. 03CH37405)*. IEEE, 2003, pp. 341–352.

[54] P. Zhou, X. Guo, P. Chen, T. Li, S. Lv, and Z. Liu, "Shortcut: Making mpc-based collaborative analytics efficient on dynamic databases," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 854–868.

[55] J. Bater, G. Elliott, C. Eggen, S. Goel, A. N. Kho, and J. Rogers, "Smcql: Secure query processing for private data networks." *Proc. VLDB Endow.*, vol. 10, no. 6, pp. 673–684, 2017.

[56] J. Liagouris, V. Kalavri, M. Faisal, and M. Varia, "{SECRECY}: Secure collaborative analytics in untrusted clouds," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 2023, pp. 1031–1056.

[57] D. Evans, V. Kolesnikov, M. Rosulek *et al.*, "A pragmatic introduction to secure multi-party computation," *Foundations and Trends® in Privacy and Security*, 2018.

[58] C. Priebe, K. Vaswani, and M. Costa, "Enclavedb: A secure database using sgx," in *2018 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2018, pp. 264–278.

[59] X. Yang, C. Yue, W. Zhang, Y. Liu, B. C. Ooi, and J. Chen, "Secudb: An in-enclave privacy-preserving and tamper-resistant relational database," *Proceedings of the VLDB Endowment*, vol. 17, no. 12, pp. 3906–3919, 2024.

[60] C. Sahin, V. Zakhary, A. El Abbadi, H. Lin, and S. Tessaro, "Taostore: Overcoming asynchronicity in oblivious data storage," in *2016 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2016, pp. 198–217.

[61] E. Stefanov and E. Shi, "Oblivistore: High performance oblivious cloud storage," in *2013 IEEE Symposium on Security and Privacy*. IEEE, 2013, pp. 253–267.

[62] E. Stefanov, M. van Dijk, A. Juels, and A. Oprea, "Iris: A scalable cloud file system with efficient integrity checks," in *Proceedings of the 28th Annual Computer Security Applications Conference*, 2012, pp. 229–238.

[63] N. Crooks, M. Burke, E. Cecchetti, S. Harel, R. Agarwal, and L. Alvisi, "Obladi: Oblivious serializable transactions in the cloud," in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 2018, pp. 727–743.

[64] N. Volgushev, M. Schwarzkopf, B. Getchell, M. Varia, A. Lapets, and A. Bestavros, "Conclave: secure multi-party computation on big data," in *Proceedings of the Fourteenth EuroSys Conference 2019*, 2019.

[65] D. Kaplan, J. Powell, and T. Woller, "Amd memory encryption," 2016.

[66] S. Pinto and N. Santos, "Demystifying arm trustzone: A comprehensive survey," *ACM computing surveys (CSUR)*, 2019.

[67] F. McKeen, I. Alexandrovich, I. Anati, D. Caspi, S. Johnson, R. Leslie-Hurd, and C. Rozas, "Intel® software guard extensions (intel® sgx) support for dynamic memory management inside an enclave," in *Proceedings of the Hardware and Architectural Support for Security and Privacy 2016*, 2016.

[68] A. Ahmad, B. Joe, Y. Xiao, Y. Zhang, I. Shin, and B. Lee, "Obfuscuro: A commodity obfuscation engine on intel sgx," in *Network and Distributed System Security Symposium*, 2019.

[69] C. Liu, X. S. Wang, K. Nayak, Y. Huang, and E. Shi, "Oblivm: A programming framework for secure computation," in *2015 IEEE Symposium on Security and Privacy*. IEEE, 2015, pp. 359–376.

[70] K. E. Batcher, "Sorting networks and their applications," in *Proceedings of the April 30–May 2, 1968, spring joint computer conference*, 1968.

[71] G. Asharov, T. H. Chan, K. Nayak, R. Pass, L. Ren, and E. Shi, "Bucket oblivious sort: An extremely simple oblivious sort," in *Symposium on Simplicity in Algorithms*. SIAM, 2020, pp. 8–14.

[72] M. Ajtai, J. Komlós, and E. Szemerédi, "An 0 (n log n) sorting network," in *Proceedings of the fifteenth annual ACM symposium on Theory of computing*, 1983, pp. 1–9.

[73] M. T. Goodrich, "Data-oblivious external-memory algorithms for the compaction, selection, and sorting of outsourced data," in *Proceedings of the twenty-third annual ACM symposium on Parallelism in algorithms and architectures*, 2011, pp. 379–388.

[74] G. Asharov, I. Komargodski, W.-K. Lin, K. Nayak, E. Peserico, and E. Shi, "Optorama: optimal oblivious ram," in *Advances in Cryptology–EUROCRYPT 2020: 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10–14, 2020, Proceedings, Part II 30*. Springer, 2020, pp. 403–432.

[75] S. Sasy, A. Johnson, and I. Goldberg, "Fast fully oblivious compaction and shuffling," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022, pp. 2565–2579.

[76] E. Stefanov, M. v. Dijk, E. Shi, T.-H. H. Chan, C. Fletcher, L. Ren, X. Yu, and S. Devadas, "Path oram: an extremely simple oblivious ram protocol," *Journal of the ACM (JACM)*, vol. 65, no. 4, pp. 1–26, 2018.

[77] X. Wang, H. Chan, and E. Shi, "Circuit oram: On tightness of the goldreich-ostrovsky lower bound," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, 2015, pp. 850–861.

[78] A. Chakraborti and R. Sion, "Concuroram: High-throughput stateless parallel multi-client oram," in *NDSS*, 2019.

[79] G. M. Adelson-Velskii and E. M. Landis, "An algorithm for the organization of information," in *Soviet Mathematics Doklady*, vol. 3, no. 1259-1262, 1962, p. 4.

[80] J. Van Den Hooff, D. Lazar, M. Zaharia, and N. Zeldovich, "Vuvuzela: Scalable private messaging resistant to traffic analysis," in *Proceedings of the 25th Symposium on Operating Systems Principles*, 2015, pp. 137–152.

[81] R. A. Popa, C. M. Redfield, N. Zeldovich, and H. Balakrishnan, "Cryptdb: Protecting confidentiality with encrypted query processing," in *Proceedings of the twenty-third ACM symposium on operating systems principles*, 2011, pp. 85–100.

[82] P. Antonopoulos, A. Arasu, K. D. Singh, K. Eguro, N. Gupta, R. Jain, R. Kaushik, H. Kodavalla, D. Kossmann, N. Ogg *et al.*, "Azure sql database always encrypted," in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, 2020, pp. 1511–1525.

[83] A. Arasu, S. Blanas, K. Eguro, R. Kaushik, D. Kossmann, R. Ramamurthy, and R. Venkatesan, "Orthogonal security with cipherbase." in *CIDR*, 2013.

[84] P. Kocher, J. Horn, A. Fogh, D. Genkin, D. Gruss, W. Haas, M. Hamburg, M. Lipp, S. Mangard, T. Prescher *et al.*, "Spectre attacks: Exploiting speculative execution," *Communications of the ACM*, 2020.

[85] M. Lipp, M. Schwarz, D. Gruss, T. Prescher, W. Haas, S. Mangard, P. Kocher, D. Genkin, Y. Yarom, and M. Hamburg, "Meltdown," *arXiv preprint arXiv:1801.01207*, 2018.

[86] B. Gedik, R. R. Bordawekar, and P. S. Yu, "Celljoin: a parallel stream join operator for the cell processor," *The VLDB journal*, vol. 18, pp. 501–519, 2009.

[87] Q. Lin, B. C. Ooi, Z. Wang, and C. Yu, "Scalable distributed stream join processing," in *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, 2015, pp. 811–825.

[88] J. Teubner and R. Mueller, "How soccer players would do stream joins," in *Proceedings of the 2011 ACM SIGMOD International Conference on Management of data*, 2011, pp. 625–636.

[89] A. Das, J. Gehrke, and M. Riedewald, "Approximate join processing over data streams," in *Proceedings of the 2003 ACM SIGMOD international conference on Management of data*, 2003, pp. 40–51.

[90] U. Srivastava and J. Widom, "Memory-limited execution of windowed stream joins," in *VLDB*, vol. 4, 2004, pp. 324–335.

[91] R. Ananthanarayanan, V. Basker, S. Das, A. Gupta, H. Jiang, T. Qiu, A. Reznichenko, D. Ryabkov, M. Singh, and S. Venkataraman, "Photon: Fault-tolerant and scalable joining of continuous data streams," in *Proceedings of the 2013 ACM SIGMOD international conference on management of data*, 2013, pp. 577–588.

[92] G. Jacques-Silva, R. Lei, L. Cheng, G. J. Chen, K. Ching, T. Hu, Y. Mei, K. Wilfong, R. Shetty, S. Yilmaz *et al.*, "Providing streaming joins as a service at facebook," *Proceedings of the VLDB Endowment*, vol. 11, no. 12, pp. 1809–1821, 2018.

[93] M. Rathee, Y. Zhang, H. Corrigan-Gibbs, and R. A. Popa, "Private analytics via streaming, sketching, and silently verifiable proofs," in *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, 2024, pp. 194–194.

[94] H. Corrigan-Gibbs and D. Boneh, "Prio: Private, robust, and scalable computation of aggregate statistics," in *14th USENIX symposium on networked systems design and implementation (NSDI 17)*, 2017, pp. 259–282.

[95] S. Addanki, K. Garbe, E. Jaffe, R. Ostrovsky, and A. Polychroniadou, "Prio+: Privacy preserving aggregate statistics via boolean shares," in *International Conference on Security and Cryptography for Networks*. Springer, 2022, pp. 516–539.

[96] R. Ostrovsky and W. E. Skeith III, "Private searching on streaming data," in *Annual International Cryptology Conference*. Springer, 2005, pp. 223–240.

[97] J. Bethencourt, D. Song, and B. Waters, "New techniques for private stream searching," *ACM Transactions on Information and System Security (TISSEC)*, vol. 12, no. 3, pp. 1–32, 2009.

[98] C. Dwork, A. Roth *et al.*, "The algorithmic foundations of differential privacy," *Foundations and Trends® in Theoretical Computer Science*, 2014.

[99] C. Dwork, M. Naor, T. Pitassi, and G. N. Rothblum, "Differential privacy under continual observation," in *Proceedings of the forty-second ACM symposium on Theory of computing*, 2010.

[100] T.-H. H. Chan, K.-M. Chung, B. Maggs, and E. Shi, "Foundations of differentially oblivious algorithms," *ACM Journal of the ACM (JACM)*, 2022.

[101] S. Chu, D. Zhuo, E. Shi, and T. H. Chan, "Differentially oblivious database joins: Overcoming the worst-case curse of fully oblivious algorithms," in *2nd Conference on Information-Theoretic Cryptography*, 2021.

[102] J. Bater, X. He, W. Ehrich, A. Machanavajjhala, and J. Rogers, "Shrinkwrap: efficient sql query processing in differentially private data federations," *Proceedings of the VLDB Endowment*, vol. 12, no. 3, 2018.

[103] S. Papadias, Z. Kaoudi, V. Pandey, J.-A. Quiané-Ruiz, and V. Markl, "Counting butterflies in fully dynamic bipartite graph streams," in *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 2024, pp. 2917–2930.

[104] K. Maliszewski, J.-A. Quiané-Ruiz, and V. Markl, "Cracking-like join for trusted execution environments," *Proceedings of the VLDB Endowment*, 2023.

[105] R. Agrawal, D. Asonov, M. Kantarcioglu, and Y. Li, "Sovereign joins," in *22nd International Conference on Data Engineering (ICDE'06)*. IEEE, 2006, pp. 26–26.

[106] Y. Wang and K. Yi, "Secure yannakakis: Join-aggregate queries over private data," in *Proceedings of the 2021 International Conference on Management of Data*, 2021.

[107] MongoDB. (2024) Queryable encryption. [Online]. Available: https://www.mongodb.com/docs/manual/core/queryable-encryption/

[108] A. Arasu, K. Eguro, M. Joglekar, R. Kaushik, D. Kossmann, and R. Ramamurthy, "Transaction processing on confidential data using cipherbase," in *2015 IEEE 31st International Conference on Data Engineering*. IEEE, 2015, pp. 435–446.

[109] S. Bajaj and R. Sion, "Trusteddb: a trusted hardware based database with privacy and data confidentiality," in *Proceedings of the 2011 ACM SIGMOD International Conference on Management of data*, 2011.

[110] Y. Sun, S. Wang, H. Li, and F. Li, "Building enclave-native storage engines for practical encrypted databases," *Proceedings of the VLDB Endowment*, 2021.

[111] D. Vinayagamurthy, A. Gribov, and S. Gorbunov, "Stealthdb: a scalable encrypted database with full sql query support," *Proceedings on Privacy Enhancing Technologies*, 2019.

[112] Microsoft. (2016) Always encrypted with secure enclaves. [Online]. Available: https://learn.microsoft.com/en-us/sql/relational-databases/security/encryption/always-encrypted-enclaves

[113] K. Maliszewski. (2026) Oblivious stream joins. [Online]. Available: https://github.com/kai-chi/obliv-stream-join

[114] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to algorithms*. MIT press, 2022.

[115] E. Shi, "Path oblivious heap: Optimal and practical oblivious priority queue," in *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2020, pp. 842–858.

[116] N. Ngai, I. Demertzis, J. G. Chamani, and D. Papadopoulos, "Distributed & scalable oblivious sorting and shuffling," in *2024 IEEE Symposium on Security and Privacy (SP)*, 2024.

[117] P. Roy, J. Teubner, and R. Gemulla, "Low-latency handshake join," *Proceedings of the VLDB Endowment*, vol. 7, no. 9, pp. 709–720, 2014.

[118] Intel. (2025) Intel xeon silver 4309y processor. [Online]. Available: https://www.intel.com/content/www/us/en/products/sku/215275/intel-xeon-silver-4309y-processor-12m-cache-2-80-ghz/specifications.html